# Comparison of YOLO and Classical Vision for Color Block Classification

Junxian Zhu
*Department of Mechanical Engineering and Materials Science*
*Duke University*

Jesen Tanadi
*Department of Civil and Environmental Engineering*
*Duke University*

Steven Yang
*Department of Mechanical Engineering and Materials Science*
*Duke University*

Chongwen Zhao
*Department of Electrical and Computer Engineering*
*Duke University*

***Abstract*—Accurate color classification is essential in robotic tasks that rely on interpreting color-coded objects, such as in board games like Mastermind. While modern deep-learning detectors such as You Only Look Once (YOLO) provide robust performance across diverse lighting and background conditions, they introduce significant computational overhead, require sizable labeled datasets, and are often unnecessary for tasks with limited visual variability. In contrast, lightweight classical computer vision techniques (such as HSV thresholding, contour extraction, and rule-based filtering) offer sufficient accuracy when the problem domain is well constrained and the scene can be reasonably controlled. In this work, we compare YOLO-based detection with a traditional OpenCV pipeline in the context of a Mastermind-playing robot. Our findings show that although YOLO is capable of solving the task, a properly tuned classical vision pipeline achieves comparable accuracy with far lower computation, simpler deployment, and no training data requirements. These results suggest that for many structured tabletop robotics problems, lightweight classical methods remain a practical and efficient choice.**

## I. Introduction

Robotic manipulation tasks often rely on accurate visual perception to identify, track, and classify objects in structured environments. While deep learning has become a dominant approach in computer vision, many robotic applications still operate in constrained domains where object appearance, background, and geometry are well-defined. In these settings, classical computer vision techniques remain attractive due to their low computational cost, transparent behavior, and ease of deployment on low-power hardware.

Conventional computer vision methods, such as HSV thresholding, contour extraction, and geometric filtering [1], are typically regarded as brittle under large illumination changes. However, when the task is narrowly scoped and the scene layout is relatively consistent, these techniques can be highly effective and require minimal computational resources. This raises an important question for robotics practitioners: when is a heavy deep-learning model like You Only Look Once (YOLO) necessary, and when is a lightweight classical pipeline sufficient?

Deep-learning approaches such as YOLO [2] offer robustness to real-world variability, but they also come with trade-offs. Often, they require annotated datasets, incur higher inference costs, and can complicate on-robot debugging when predictions fail. In contrast, classical OpenCV [3] pipelines can be tuned quickly, run efficiently on CPUs, and avoid dataset collection entirely. These considerations are particularly relevant in tabletop tasks like Mastermind, where objects are simple, well separated, and have highly distinguishable colors.

In this work, we examine traditional OpenCV-based pipelines and compare it to YOLO-based deep learning through the lens of a robotic system playing a Mastermind-inspired game. In our setup, a robot places colored blocks onto a staging area where a camera interprets each guess. Accurate and consistent color classification is essential for game correctness and robot decision-making, making the task an ideal benchmark for evaluating perception methods. We compare the two approaches with respect to robustness and sensitivity to scene variations, and overall development effort.

## II. Background

In this section, we provide the necessary background for understanding the problem setting and the vision techniques evaluated in this work. We begin by outlining our adaptation of the Mastermind game and explaining why it serves as an effective benchmark for robotic color-classification tasks. We then review traditional color-segmentation approaches commonly implemented with OpenCV, followed by an overview of YOLO-based deep-learning methods that offer a modern alternative for robust object detection and classification.

### A. The Mastermind Benchmark

Mastermind is a logic game where a codebreaker deduces a secret color sequence chosen by a codesetter. In robotics, this task serves as a benchmark for "color constancy": the ability to correctly classify objects despite variations in illumination and viewpoint. Unlike controlled simulations, physical deployments introduce environmental noise, such as varying light conditions, shadows, and specular reflections, which challenge the reliability of vision systems.

### B. Classical Color Segmentation

Although these methods are sometimes characterized as brittle, we find that in a controlled tabletop environment such as our Mastermind setup, carefully selected Hue, Saturation, and Value (HSV) ranges combined with simple morphological filtering provide stable and repeatable results. The block colors are distinct, the staging area is uniform, and the camera viewpoint is fixed. Under these conditions, classical techniques excel. Rather than being a limitation, the explicitness of the pipeline makes errors easy to diagnose and correct, offering a level of transparency that deep-learning models typically lack.

### C. Deep Learning and YOLO Architecture

Although YOLO performs fairly well on this task, we observe that its advantages are not essential for a structured environment like our tabletop setup. As illustrated in Figure 1, the YOLO architecture is substantially more complex than what is required for simple color discrimination among well-separated blocks. Once lighting is moderately controlled and the background is uniform, the classical OpenCV pipeline achieves accuracy comparable to YOLO without requiring any training data, GPU resources, or hyperparameter optimization. The additional architectural sophistication of YOLO therefore provides little practical benefit in this context, making it a viable, but not strictly necessary solution for this problem domain.

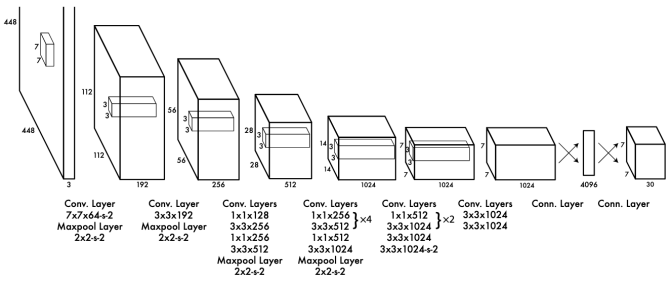


Fig. 1. YOLO architecture showing the cascade of convolutional and fully connected layers that make up the model. Image credit: [2].

## III. Methods

In this section, we present the physical configuration of the Mastermind robotic system and the components involved in its perception workflow. We then detail the manual tuning of the OpenCV pipeline, which requires iterative adjustments to lighting and color-threshold parameters. Finally, we describe the dataset creation and training process used to develop our YOLO model.

### A. Hardware & Staging Area Setup

The physical system consists of a Kinova Gen3 Lite robotic arm and a Logitech RGB webcam arranged to monitor a designated staging area (the region where the robot places its color-coded guesses). The game pieces are 3D-printed blocks, each produced by printing distinct color on paper and affixing them to the block surfaces. The colors used are black, red, purple, blue, yellow, and green.

The experimental setup is located adjacent to a large garage-style door with multiple exterior-facing windows. Even though the room is brightly top-lit, this placement results in variable lighting conditions throughout the day, including shifts in brightness, shadows, and color temperature. This condition is illustrated in Figure 2.

Such environmental changes significantly increase the difficulty of the vision task. Compounding this challenge, the printed paper textures reduce the saturation and uniformity of the block colors, making both classification and segmentation more demanding.

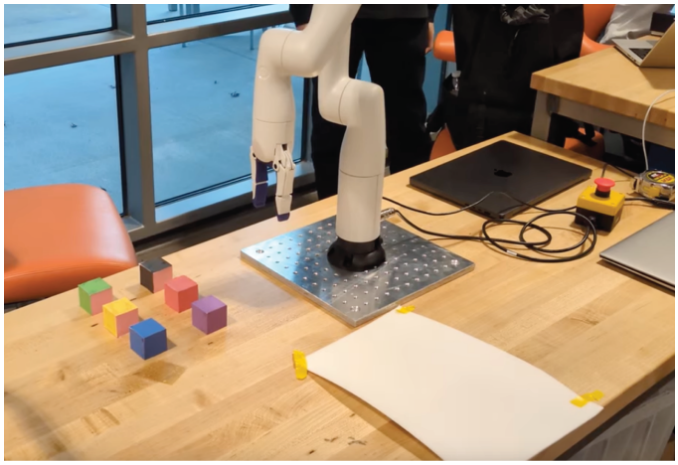

Fig. 2. The physical setup of our benchmarking experiments, showing the color block storage area (left of the robot arm) and the white staging area (lower right). The RGB camera, positioned on a tripod next to the staging area, is not visible in this view. Large exterior windows in the background introduce natural illumination changes throughout the day.

### B. Traditional OpenCV Pipeline

As shown in Figure 3, the traditional computer vision approach relies on a sequence of handcrafted operations to isolate and classify block colors [1]. First, the camera image is cropped and converted from RGB to HSV color space, which provides better separation of chromatic information under moderate lighting variation. Each color category is then isolated using predefined HSV threshold ranges. To reduce noise, the resulting binary masks undergo smoothing and morphological operations, after which contours are extracted to identify candidate block regions. The centroids of these contours are computed and labeled according to the mask from which they originated, completing the classification step. Below we outline the specific methods we take to segment and classify images:

1) **Crop original image** to remove extraneous background elements that may lead to erroneous segmentation. We choose a tight crop to focus the content of the image on the blocks.
2) **Increase image saturation** to enhance the separability of the color blocks. This step compensates for the low-

TABLE I
HSV THRESHOLD RANGES FOR EACH COLOR

| Color | Lower HSV Range | Upper HSV Range |
|---|---|---|
| red | (0, 100, 100) | (10, 255, 255) |
| red2 | (170, 100, 100) | (180, 255, 255) |
| green | (40, 90, 100) | (90, 255, 255) |
| blue | (100, 200, 100) | (130, 255, 255) |
| yellow | (10, 110, 100) | (35, 255, 255) |
| purple | (130, 100, 100) | (160, 200, 255) |
| black | (0, 0, 0) | (120, 130, 60) |

saturation images produced by the lighting conditions and the limitations of the camera hardware, which otherwise result in dull or muted colors.

3) **Convert images to masks** using manually defined HSV ranges, applied separately for each color. For example, yellow spans $(10, 110, 100)$ to $(35, 255, 255)$, while black spans $(0, 0, 0)$ to $(120, 130, 60)$. Ranges for all colors are shown in Table I. Note that the two entries for red (red and red2) account for the cyclical nature of the hue channel in HSV.
4) **Erode and dilate masks** to remove small noisy regions and smooth the segmented areas. Erosion eliminates isolated pixels and minor artifacts, while dilation restores the main regions to their original size. Together, these operations produce cleaner masks in which the largest contour is more likely to correspond to the block of the target color.
5) **Choose largest region** by employing manually set thresholds. This step removes smaller regions of the mask that may have been produced by incorrect segmentation or noise.
6) **Label the masked regions** and sort the labels according to the $x$-coordinate of their centroids, such that the order of the labels corresponds to the color sequence of the blocks in the image.

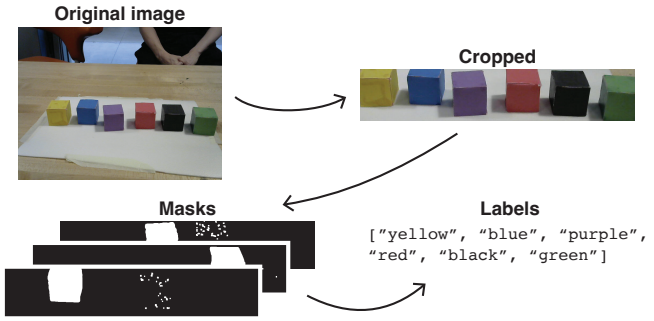


Fig. 3. Overview of the OpenCV segmentation process, showing how an image obtained from the Logitech webcam is cropped, masked, segmented, and converted to a list of colors.

While straightforward to implement, this pipeline presents some challenges. The primary challenge is the requirement to manually tune the HSV ranges to account for lighting variability. Even small environmental changes (e.g., shadows cast by the robot arm, glare on block surfaces, or fluctuations in camera white balance) can cause degradation in detection accuracy. As a result, the OpenCV method demands consistency in the physical setup.

### C. YOLO-based Color Classification

For the deep-learning approach, we begin with a pre-trained YOLOv8 model [4] and fine-tune it using approximately 4,000 additional images of our colored blocks. These training images are collected from video stills recorded directly in the deployment environment, ensuring coverage of the staging area under a variety of lighting conditions. Each image in the dataset is annotated using the LabelImg tool [5], providing bounding boxes and color labels needed for supervised training.

Once trained, YOLO processes each frame in a single forward pass, simultaneously predicting bounding boxes around detected objects, assigning class labels corresponding to block colors, and providing confidence scores for each detection. Unlike traditional color-segmentation methods, YOLO does not rely on manually defined thresholds. Instead, it learns to recognize both color and shape directly from data, giving it strong resilience to real-world imaging conditions. The model remains robust under varying levels of brightness, shadows cast by the robot arm, and changes in background texture. Figure 4 shows a sample detection and classification by YOLO.

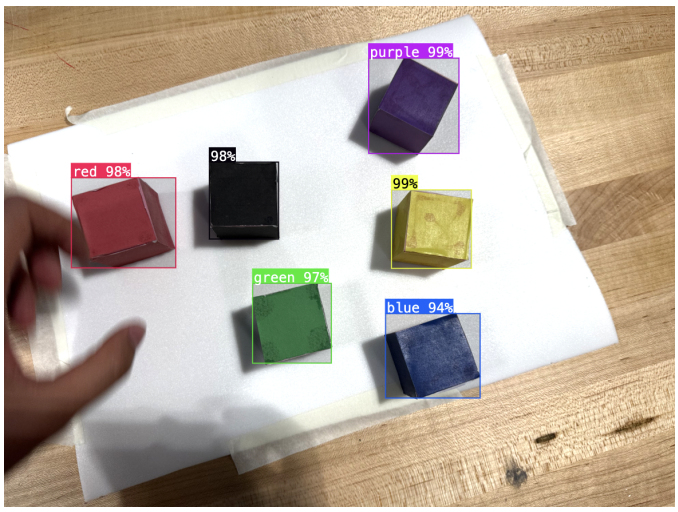


Fig. 4. A sample detection and classification result from our trained YOLO model. Each block is correctly detected and its color correctly inferred; prediction confidence (in percent) is shown above each bounding box.

## IV. RESULTS AND DISCUSSIONS

Both YOLO and classical OpenCV segmentation are evaluated across different lighting conditions and with several color combinations of the blocks. Despite YOLO's robustness to variation, we find that the classical pipeline performs comparably when the environment is moderately consistent. After initial tuning, HSV-based segmentation reliably identifies all six block colors with minimal false detections. Once the lighting is calibrated, the system requires no retraining, and run-time execution remains lightweight.

While YOLO maintains stable performance across all tests, the margin of improvement over classical methods is small relative to the additional computational cost and development overhead. YOLO's strength lies in unstructured or highly variable environments; however, in a bounded tabletop game with fixed geometry and distinct color classes, classical vision techniques demonstrate that they are fully sufficient to achieve correct game-state estimation. It is also important to note that YOLO, like all deep-learning approaches, does not achieve "perfect" performance. False negatives are possible, as illustrated in Figure 5. Unlike traditional techniques, such errors are often difficult to diagnose or correct, as the internal decision process of deep models is not easily interpretable.

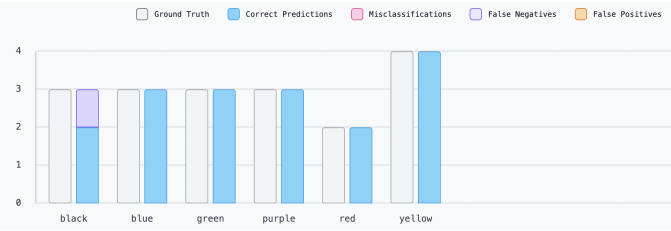


Fig. 5. Classification performance of our trained YOLO model. The false negatives (purple bar) associated with the black block highlight a recurring challenge: as with the traditional OpenCV pipeline, the black block is frequently misclassified due to its similarity to cast shadows.

In our experiments, classical OpenCV-based methods prove sufficient for reliable color classification, provided that the system undergoes careful manual calibration. The pipeline requires tuning of HSV thresholds, adjustment of morphological operations, and occasional retesting whenever lighting conditions or camera parameters change. Without these adjustments, the resulting masks may be inconsistent or noisy, as illustrated in Figure 6. Colors such as green, blue, and purple often bleed into one another, red may be partially segmented, and black frequently includes cast shadows. These artifacts highlight the primary trade-off of traditional techniques: while they achieve strong performance once tuned, they rely heavily on hand-engineered parameters and lack the robustness of learning-based approaches when conditions drift. However, once tuned, the method performs well and produces consistent labels that align with expectations. In Figure 7, we show sample image–label pairs obtained from the manually tuned OpenCV pipeline.

Admittedly, YOLO is not strictly necessary for a color-classification task as simple as the one in this application. A range of intermediate techniques exists between manual color segmentation and end-to-end deep learning. Methods such as support vector machine (SVM) or lightweight neural classifiers (e.g., a model trained on multi-hot encoded color labels) are viable candidates [6]–[8]. However, as these are supervised learning approaches, they would likely require a substantial labeled dataset to achieve robustness across lighting conditions. Our choice to test YOLO is motivated in part by its practical advantages: high-quality pre-trained models

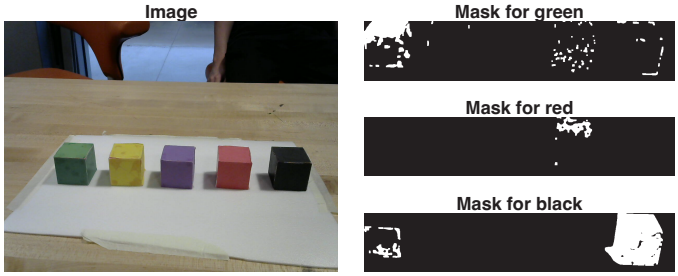


Fig. 6. Examples of faulty masks generated by the classical OpenCV pipeline. The green mask includes regions from nearby blocks, the red mask shows partial segmentation, and the black mask captures cast shadows. These issues highlight how improper thresholding leads to noisy or inconsistent masks.

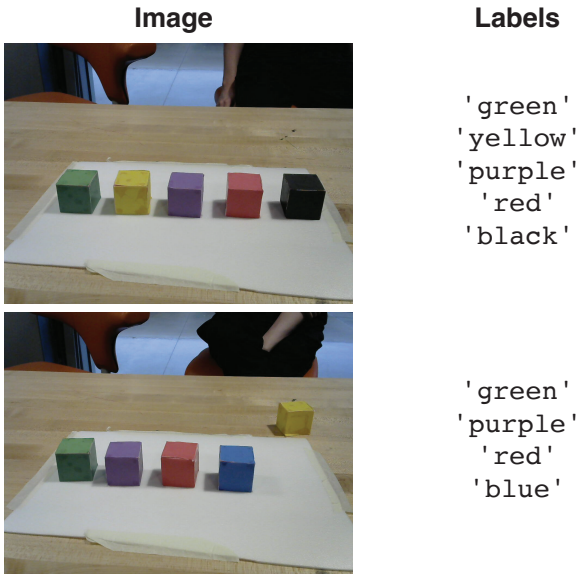


Fig. 7. Sample image–label pairs from the conventional OpenCV method. The top row shows correct classification of all blocks within the staging area. The bottom row illustrates that blocks outside the staging area (in this case, the yellow block) are correctly ignored.

are readily available, and those models can be fine-tuned efficiently for new domains.

Lastly, we compare the development time required for the two approaches. With OpenCV-based methods, most effort is spent manually tuning parameters—such as HSV ranges and morphological settings—to ensure consistent results. In contrast, using YOLO requires assembling a sizable labeled dataset, along with additional time for data augmentation, hyperparameter tuning, and model training. In our case, approximately 4,000 images were used. For a task as simple and well-structured as color-block classification, this additional development overhead makes YOLO less practical than classical computer vision techniques.

## V. Conclusion

Although YOLO-based detectors provide strong performance in general-purpose object detection and remain valuable for unstructured scenes, our findings show that they are not strictly necessary for structured tabletop robotics tasks such

as Mastermind color classification. A lightweight classical OpenCV pipeline based on HSV thresholding, contour extraction, and simple spatial heuristics achieves comparable accuracy with far lower computational cost, zero training data requirements, and greater transparency during debugging.

These results illustrate an important design principle: deep learning should not be adopted by default when simpler solutions are sufficient. For tasks involving distinct objects, controlled backgrounds, and minimal occlusion, classical methods remain practical, efficient, and highly effective. Future work may explore hybrid pipelines that retain the computational efficiency of classical vision while introducing learning-based components only where they provide clear benefits.

Finally, we note that the choice between classical and learning-based perception should be guided by the structure of the task and the practical constraints of the system. As robotic platforms continue to diversify, understanding when lightweight methods suffice remains an important consideration for efficient and reliable deployment.

## REFERENCES


[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016, pp. 779–788.

[3] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[4] Roboflow, "Yolov8 model," https://roboflow.com/model/yolov8, accessed: 2025-12-05.

[5] HumanSignal, "labelimg," https://github.com/HumanSignal/labelImg, accessed: 2025-12-05.

[6] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[7] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[8] M.-L. Zhang and Z.-H. Zhou, "A review on multi-label learning algorithms," *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 8, pp. 1819–1837, 2014.